

01_story_pitches

Helios — Story & Elevator Pitches

Section 1: Project Story

The same project, told five ways. Each version is tuned to one audience's attention budget and the one thing that audience is actually deciding. Use the version that matches the room; do not blend them.

Resume (1-2 lines)

Helios — Autonomous AI Growth Analyst (BigQuery · dbt · Claude Agent SDK · MCP). Built a self-running, seven-agent diagnosis engine that distinguishes mix-shift from rate-change, prices funnel movement as dollars of revenue-at-risk, and prescribes a power-analyzed experiment backlog from governed SQL — **85%+ root-cause accuracy vs a 45% naive baseline**, graded automatically in CI, with **0 hallucinated columns**.

Recruiter screen (~30 seconds, plain language)

"Helios is an AI system I built that acts like an always-on growth analyst for an online store. When a sales funnel moves — say checkout conversion drops — a human analyst normally spends one to three days figuring out *why*. Helios does it in under five minutes, on a schedule, without anyone asking. It figures out the real reason behind a number changing, puts a dollar value on it, and recommends the one experiment most worth running. I built it on Google's public e-commerce analytics dataset using BigQuery, dbt, and Claude's agent framework. The headline result: it gets the root cause right 85% of the time versus 45% for the obvious naive approach, and I prove that with an automated test suite that runs on every code change."

Technical interview (~2 minutes, architecture + trust)

"Helios is an autonomous growth-diagnosis engine. The product loop is: detect an anomaly in a funnel metric, decompose *why* it moved, diagnose the root cause, verify it adversarially, size a fix, and ship an executive Decision Brief — all on a schedule.

The technical centerpiece is a **mix-shift versus rate-change decomposition**. Any aggregate rate $R = \sum w_i \cdot r_i$ over segments, where w is each segment's volume share and r its rate. The change ΔR splits exactly into $\text{mix} = \sum \Delta w_i \cdot r_i(t_0)$, $\text{rate} = \sum w_i(t_0) \cdot \Delta r_i$, and an interaction term. That tells you whether conversion fell because in-segment behavior got worse, or merely because traffic composition shifted toward lower-converting segments. It's how I dissolve Simpson's paradox — the aggregate can fall while every segment rises.

The trust story is three structural pillars. First, **governed-SQL grounding**: the LLM never authors raw SQL. It composes governed metrics through a semantic layer — `semantic-mcp` is the *only* path to SQL — so hallucinated columns are structurally impossible, not just discouraged. Second, **deterministic statistics**: all decomposition, significance testing, power, and forecasting run in real Python via `stats-mcp`; nothing statistical is generated in token-space. Third, **adversarial verification plus offline eval**: a Critic agent tries to refute every finding — mix confound, insufficient sample, seasonality, data-quality artifact — and a 34-scenario labeled benchmark grades root-cause accuracy in CI on every commit.

Architecturally that's five MCP servers as a capability/trust boundary and seven agents on a deterministic finite-state machine. The LLM composes tool calls; it never controls flow."

Hiring manager round (~2 minutes, impact + collaboration + decisions)

"The problem Helios attacks is the insight-to-action gap. Every growth org has dashboards that tell you *what* happened, but the expensive question is *why*, and that still routes to a human analyst for one to three days per anomaly. That analyst is a serial, manually-triggered bottleneck — and they routinely confuse a traffic-mix change with a real behavior change, which funds the wrong fix.

Helios inverts that. Diagnosis becomes continuous and proactive: every scheduled run re-derives the funnel, flags what moved, separates mix from rate, prices it in revenue-at-risk, and ships a brief the head of growth can forward to the CEO — *before* anyone asks. I designed it around four real personas: the Head of Growth who owns the revenue number, the PM who owns a funnel surface, the analyst who's tired of being the human RCA oracle, and the founder who just wants the three things that moved revenue and the plan.

The decision I'm proudest of is making trust *measurable* instead of asserted. 'The AI says so' isn't shippable, so I built a labeled benchmark and wired it into GitHub Actions: every commit grades diagnosis accuracy, and a regression that drops top-1 accuracy by more than two points or introduces any hallucination fails CI. The hardest tradeoff was scope discipline — it would have been easy to drift into a dashboard or a chatbot. I held a hard anti-product line: conversation is a secondary drill-down surface; the heartbeat is the autonomous run."

Founder round (~2 minutes, business + ROI + what next)

"Helios creates a category rather than competing in one. Descriptive BI tells you *what*; conversational text-to-SQL tools answer one human-prompted question at a time and frequently hallucinate; product analytics gives you funnels with some *who*. None of them autonomously answer *why*, *how much in dollars*, and *what to do next*. That white space — autonomous, governed, statistically-defensible, dollar-quantified diagnosis — is the wedge.

The ROI math is direct. Manual root-cause analysis costs one to three analyst-days per anomaly, and a senior analyst's funnel is dozens of metrics across many segments every week. Helios takes that to under five minutes per run on a fixed BigQuery byte budget — and, more importantly, it stops misdiagnosis. Every week a depressed conversion rate goes undiagnosed is compounding lost revenue; every wrong mix-vs-rate call funds the wrong engineering work. Helios converts that invisible liability into a visible, prioritized, dollar-denominated backlog.

The wedge is deliberately narrow so it can be excellent and verifiable, but the expansion path is clear: multi-tenant productionization, warehouse-agnostic adapters behind the semantic layer, and eventually true causal inference — difference-in-differences and uplift modeling — plus a closed loop that pushes experiment cards to a testing platform, reads back the result, and updates the backlog. I'm honest that the public dataset is observational and three months long, so today Helios *designs and sizes* experiments and runs quasi-experimental readbacks rather than live A/Bs. The architecture is built to absorb live experimentation the moment the data supports it."

Why Helios exists — the core arguments

Why Helios exists. Modern e-commerce orgs are drowning in descriptive signal and starving for diagnostic answers. A dashboard can show `session_conversion_rate` fell from 2.1% to 1.7%, but cannot say why, cannot price it, and cannot prescribe a fix. Helios exists to close that insight-to-action gap — to make the question "why did the funnel move?" answerable by an artifact that already exists, is grounded in governed SQL, is defended statistically, and has survived an adversarial critique, rather than by a Slack ping that kicks off two days of ad-hoc querying.

What problem it solves. It automates root-cause analysis of the conversion funnel and makes every finding actionable. Concretely: detect the anomaly, decompose mix vs rate vs interaction, verify the root cause against governed data, quantify revenue-at-risk in dollars, and prescribe a powered experiment — collapsing a one-to-three-day manual process to under five minutes, while structurally preventing the two failure modes that wreck manual RCA: hallucinated metrics and confusing composition change with behavior change.

Why dashboards are insufficient. Dashboards report *what*, never *why* — no causal-style attribution, no mix/rate split. They are pull, not push: someone must notice and ask. They are blind to interaction confounds across dimensions like `channel_group × device_category`. They carry no dollar quantification, so a percentage-point drop never becomes a budget decision. And they prescribe nothing — no experiment, no power analysis, no prioritized backlog. A dashboard is the *input* to diagnosis, not diagnosis.

Why autonomous diagnosis matters. A human analyst is a serial, reactive oracle queried only after someone notices a number looks off — which can be days later. Autonomy inverts that latency: diagnosis becomes a continuous background process that runs whether or not anyone is watching, the way CI runs on every commit. Proactive beats reactive because the cost of an undiagnosed week compounds, and because a scheduled, governed, repeatable run is something a team can *trust* in a way they can never trust an ad-hoc one-off.

Why MCP was chosen. Model Context Protocol lets me split capability across servers and enforce hard boundaries the LLM cannot cross. `semantic-mcp` is the *only* path to SQL and `stats-mcp` the *only* path to math, with per-agent tool allow-lists — the Narrator literally cannot call `run_query`. That capability isolation is the trust argument made structural: hallucinated columns and hand-computed statistics aren't policed after the fact, they're impossible by construction. MCP turns "please don't write bad SQL" into "you cannot write SQL at all."

Why an agent architecture was chosen. Diagnosis is genuinely multi-step and benefits from specialization and adversarial review — a single monolithic prompt can't both confidently propose a root cause *and* rigorously refute it. So I use seven agents on a deterministic finite-state machine: the LLM composes tool calls at each node, but the *flow* is code, not model-controlled. That gives me the reasoning flexibility of agents (a best-first hypothesis tree, an adversarial Critic that can downgrade or drop a finding) with the reproducibility and cost control of a fixed pipeline — Opus for orchestration and critique where judgment matters, Sonnet for high-volume sub-tasks.

Section 2: Elevator Pitches

Five ready-to-speak scripts. Each tier strictly adds depth on top of the previous one — pick by the time you actually have.

30 seconds — thesis + wedge

"Helios is an autonomous AI Growth Analyst. Instead of another 'ask your data' chatbot, it runs on a schedule and proactively diagnoses *why* an e-commerce funnel moved — and it nails the one

thing analysts get wrong: it separates a real behavior change from a mere shift in traffic mix. Every finding ships with a significance test, a dollar revenue-at-risk number, and a recommended experiment. On a labeled benchmark it hits 85%+ root-cause accuracy versus 45% for the naive approach."

1 minute — + the problem + how it works at a glance

"Helios is an autonomous AI Growth Analyst. The problem it solves: when a funnel metric moves, a human analyst spends one to three days finding the root cause — and routinely confuses *mix-shift* (traffic composition changed) with *rate-change* (in-segment behavior changed), which is Simpson's paradox and funds the wrong fix.

Helios runs on a schedule and, in under five minutes, does the whole loop: detect the anomaly, decompose the move into mix versus rate versus interaction, diagnose the root-cause segment, price it in dollars of revenue-at-risk, and prescribe a powered experiment — then ships an executive Decision Brief before anyone asks. The decomposition is the centerpiece: it tells you whether conversion dropped because behavior got worse or because your traffic mix shifted toward lower-converting segments. On a labeled benchmark, 85%+ root-cause accuracy versus a 45% naive baseline."

3 minutes — + architecture + the trust pillars

"[Open with the 1-minute pitch, then continue:]

Under the hood it's five MCP servers and seven agents. The five MCP servers are the trust boundary. `warehouse-mcp` is the sole BigQuery client, with a mandatory dry-run byte-budget gate before any query and a reconcile check against canonical totals. `semantic-mcp` is the *only* path to SQL — it composes validated queries from a governed registry of 28 metrics and 16 dimensions, so the model references metric *names* like `view_to_cart_rate`, never raw columns. `stats-mcp` is the *only* path to math — decomposition, significance, forecasting, cohort, RFM. `experiment-mcp` does power analysis and experiment design. `report-mcp` renders the brief and handles memory.

The seven agents run on a deterministic state machine: an Orchestrator on Opus; a Monitor that detects anomalies; a Decompose agent that runs the mix/rate math; a Diagnose agent on Opus that builds a best-first hypothesis tree and SQL-verifies each branch; a Critic on Opus that adversarially tries to refute every finding; a Prescribe agent that builds experiment cards; and a Narrator that writes the brief. The LLM composes the tool calls — it never controls the flow.

That gives me three defensibility pillars. One, **governed-SQL grounding** — the semantic layer is the only SQL author, so hallucinated columns are structurally impossible. Two, **deterministic**

statistics — all math is real code, never generated tokens. Three, **adversarial verification plus offline eval** — the Critic refutes findings, and a labeled benchmark grades accuracy in CI. The thesis is that the hard part of an AI analyst isn't generating insights; it's making them correct and *trusted*."

5 minutes — + the eval, results, and honest tradeoffs

"[Open with the 3-minute pitch, then continue:]

The reason I can claim 85% isn't a vibe — it's an offline labeled benchmark. The public GA4 dataset has real anomalies but they're *unlabeled*, so I can't grade against them. Instead I clone a baseline window into an eval dataset and surgically inject anomalies with a known ground truth, then make the pipeline rediscover what I hid. A *rate* perturbation multiplies one segment's conversion rate while holding its volume fixed — ground truth is rate-change. A *mix* perturbation multiplies a segment's sessions while holding rates fixed — ground truth is mix-shift. Because totals are conserved, the analytic mix/rate/interaction split is exact and becomes the gold target.

The benchmark is 34 scenarios across seven buckets: single- and multi-segment rate changes, single- and multi-segment mix shifts, seasonality decoys the system must *not* flag, no-anomaly controls that test false-positive rate, and data-quality artifacts the Critic must catch as data issues rather than behavior. The contract: top-1 root-cause accuracy $\geq 85\%$ versus $\leq 45\%$ for a naive 'largest absolute segment delta' baseline; decomposition error $\leq 10\%$ MAPE; 0 hallucinated columns; 100% of findings carrying a significance test and a dollar figure; under five minutes per run; and cost under a fixed 5 GiB BigQuery byte budget. CI gates regressions on every PR.

Now the honest tradeoffs, because they read as maturity. The dataset is observational and historical — about three months, late 2020 — so Helios *designs and sizes* experiments and runs quasi-experimental readbacks like difference-in-differences; it does not run live A/Bs. That short window means true LTV and long-horizon retention are deferred to short-horizon proxies with stated assumptions. `user_id` is almost always null, so identity is cookie-grain and user-level metrics are explicit cookie approximations. And there's a classic GA4 gotcha: event-level `traffic_source` is *first-touch*, not session source — I derive session-scoped source/medium and fall back to first-touch only when null. I'd rather state these limits plainly than have an interviewer find them."

10 minutes — deep dive: one autonomous run end-to-end, decisions, limits, roadmap

"[Open with the 5-minute pitch, then walk one run end-to-end:]

One run, end to end. The scheduler fires. The **Monitor** agent pulls a canonical series — say `session_conversion_rate` — through `semantic-mcp` (which composes the SQL) and `warehouse-mcp` (which dry-runs the cost, executes, and reconciles to within 0.5%), then calls `stats-mcp.detect_anomaly`. It flags that conversion fell week-over-week. The **Decompose** agent then calls `stats-mcp.decompose_change` across canonical dimensions like `channel_group` and `device_category`. Suppose the result is: per-segment rates are essentially flat, but there's a large *mix* effect — a surge of low-converting mobile, Paid Social traffic shifted the weights. That's the Simpson's-paradox moment: the naive read was 'checkout is broken'; the decomposition says behavior didn't change at all, the mix did.

The **Diagnose** agent (Opus) takes that and builds a best-first hypothesis tree, verifying each branch with governed SQL — confirming the mobile/Paid Social volume spike is real and localizing it. It dollarizes: affected sessions × rate delta × revenue-per-session, attributed by segment, yields revenue-at-risk. Then the **Critic** (Opus) attacks the finding: is this a mix confound? Is the sample big enough for significance? Could it be a known seasonal swing — post-holiday in this dataset? Is it a data-quality artifact like a NULL spike or a late shard? It issues a verdict — PASS, DOWNGRADE, or DROP. Only survivors proceed. The **Prescribe** agent (Sonnet) sizes the fix — here, an acquisition-mix intervention, not a checkout fix — via `experiment-mcp.power_analysis` and `runtime_estimate`. Finally the **Narrator** renders the Decision Brief through `report-mcp`, and the diagnosis is saved to memory so the next run can recall it and track whether the recommended action actually worked. Findings hand off between agents as a typed JSON 'Finding' envelope.

Key design decisions and the alternatives I rejected. First, a deterministic state machine over a free-roaming agent — I traded some autonomy for reproducibility, cost control, and the ability to put a hard Critic gate in the flow. Second, MCP capability isolation over prompt instructions — 'you may not call `run_query`' enforced by tool allow-lists beats 'please write good SQL,' so the Narrator structurally cannot touch the warehouse. Third, an Opus/Sonnet split — Opus where judgment matters (orchestrate, diagnose, critique), Sonnet for high-volume sub-tasks, to keep cost sane. Fourth, deterministic stats in real Python over LLM-computed math — the decomposition is unit-tested against hand-worked golden values, because a keystone transform that's silently wrong corrupts every downstream number. Fifth, the offline injection benchmark over eyeballing live anomalies — it's the only way to get labeled ground truth and a defensible accuracy number.

Limitations, stated honestly. Observational, three-month, late-2020 data: experiments are designed and quasi-experimentally read back, not run live; LTV and long retention are proxied; identity is cookie-grain because `user_id` is null; channel attribution navigates the first-touch `traffic_source` gotcha. The decomposition is correlational, not causal — which is exactly why the Critic attaches explicit confound caveats rather than overclaiming.

Roadmap. Phase 3 is productionization: multi-tenant isolation, near-real-time streaming ingestion, and warehouse-agnostic adapters behind `warehouse-mcp` so the semantic layer stays the only SQL author across Snowflake, Databricks, or DuckDB. Phase 4 is the frontier: replace correlational decomposition with true causal inference — difference-in-differences, synthetic control, double-ML — where data supports it; join GA4 with CRM and cost data for true CAC and ROAS; and close the loop by pushing experiment cards to a real experimentation platform, reading results back, and auto-updating the backlog. The architecture was built to absorb each of those without touching the trust spine."